

SQLite Notes App

Technical Report

Flutter • SQLite • sqflite • CRUD • Local Persistence

Platform	Flutter (Dart)
Database	SQLite via sqflite
Version	1.0.0+1
Date	May 29, 2026

Prepared as part of Flutter SQLite Learning Module

1. Executive Summary

The **SQLite Notes App** is a fully functional Flutter application that demonstrates local data persistence using SQLite through the **sqflite** package. Developed as a practical learning deliverable for the Flutter SQLite module, the app covers every key objective: database initialization, table creation, full CRUD operations, JOIN queries, search functionality with WHERE clauses, and cross-platform desktop support via **sqflite_common_ffi**.

The application manages notes organized into categories, providing a real-world one-to-many relationship scenario. Users can create, view, edit, and delete notes; search across titles and content in real time; and filter by category — all with data that persists across app restarts.

Key Achievement

All nine SQL operations from the learning objectives are implemented and exercised: CREATE TABLE, INSERT, SELECT, UPDATE, DELETE, WHERE, ORDER BY, LIMIT/OFFSET, and JOIN.

2. Project Overview

2.1 Learning Objectives Met

The following table maps each learning objective to its implementation status in the app:

Objective	Implementation	Status
Setup SQLite in Flutter	sqflite + path packages; DatabaseHelper singleton; sqflite_common_ffi for desktop	Complete
Perform CRUD operations	insertNote, getNotes, getNoteById, updateNote, deleteNote methods in DatabaseHelper	Complete
Query database	rawQuery with JOIN, WHERE, ORDER BY, LIMIT, OFFSET; helper query() for simple reads	Complete
Handle relationships	categories (1) to notes (many); FOREIGN KEY with ON DELETE RESTRICT; JOIN in SELECT	Complete
Search functionality	LIKE query on title and content with 250 ms debounce; category chip filter	Complete
Data persistence	Database stored in app documents directory; survives app restarts	Complete
Cross-platform support	sqflite_common_ffi initializes FFI factory on Windows, macOS, and Linux	Complete

2.2 Technology Stack

Item	Detail
Language	Dart 3.x (SDK ^3.11.5)
Framework	Flutter (Material 3)
Database	SQLite — accessed via sqflite ^2.4.2+1
Path utility	path ^1.9.1
Desktop support	sqflite_common_ffi ^2.4.0+3
Lint rules	flutter_lints ^6.0.0
App version	1.0.0+1

3. Application Architecture

3.1 Directory Structure

The project follows a clean, layered structure separating data, models, UI pages, and platform-specific initialization:

```
sqlite_app/
  lib/
    data/
      database_helper.dart    <- Singleton DB access & all SQL logic
    models/
      note.dart              <- Note data model with toMap/fromMap
      note_category.dart     <- Category model
    pages/
      notes_page.dart        <- Full UI: list, search, filter, CRUD
    src/
      database_initializer.dart  <- Conditional export router
      database_initializer_io.dart <- sqflite_common_ffi (desktop)
      database_initializer_stub.dart <- No-op stub (mobile/web)
    main.dart                <- App entry point
  pubspec.yaml
  README.md
```

3.2 Layer Responsibilities

- Data layer (`database_helper.dart`): All SQL is centralized here. The UI never writes raw SQL — it calls typed methods like `insertNote()` or `getNotes()`.
- Model layer (`note.dart`, `note_category.dart`): Immutable Dart classes with `toInsertMap()`, `toUpdateMap()`, and factory `fromMap()` constructors for safe serialization/deserialization.
- UI layer (`notes_page.dart`): `StatefulWidget` consuming the `DatabaseHelper` singleton. Triggers reloads after mutations; debounces search input.
- Platform layer (`src/`): Conditional import pattern selects the right database factory at compile time — no runtime `if/else` in `main`.

3.3 Singleton Pattern

`DatabaseHelper` uses a private constructor and a static instance field, ensuring a single shared object across the entire app lifecycle:

```
class DatabaseHelper {
  DatabaseHelper._(); // private constructor
  static final DatabaseHelper instance = DatabaseHelper._();

  Future<Database>? _database;
```

```
Future<Database> get database async {  
    _database ??= _openDatabase(); // lazy initialization  
    return _database!;  
}  
}
```

The lazy ??= initialization means the database file is only opened on the first access, not at construction time. The Future is cached so concurrent callers await the same opening operation.

4. Database Design

4.1 Schema

The database contains two tables forming a one-to-many relationship: one category can have many notes, but every note must belong to exactly one category.

```
CREATE TABLE categories (
  id      INTEGER PRIMARY KEY AUTOINCREMENT,
  name    TEXT NOT NULL UNIQUE
);

CREATE TABLE notes (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  title         TEXT NOT NULL,
  content       TEXT NOT NULL,
  category_id   INTEGER NOT NULL,
  created_at    TEXT NOT NULL,
  updated_at    TEXT NOT NULL,
  FOREIGN KEY (category_id) REFERENCES categories (id)
  ON DELETE RESTRICT
);
```

4.2 Design Decisions

Decision	Rationale
AUTOINCREMENT on ids	Prevents id reuse after deletions — important for referential integrity.
TEXT NOT NULL UNIQUE on category.name	Enforces unique category names at the database level; INSERT with ConflictAlgorithm.ignore silently skips duplicates during seeding.
Timestamps as ISO-8601 TEXT	SQLite has no native DATETIME type. Storing as ISO-8601 strings preserves timezone information and sorts lexicographically.
FOREIGN KEY with ON DELETE RESTRICT	Prevents orphaned notes if a category is removed. PRAGMA foreign_keys = ON is set in _configureDatabase() since SQLite disables FK enforcement by default.
Separate toInsertMap / toUpdateMap	Insert must not include id (auto-generated); update must not overwrite created_at. Two maps enforce this separation at the type level.

4.3 Database Initialization & Seeding

On first launch the `_createDatabase` callback runs, creating both tables and then seeding four default categories (Personal, Work, Study, Ideas) using a batch insert for efficiency:

```
Future<void> _seedCategories(Database db) async {  
  final batch = db.batch();  
  for (final category in _defaultCategories) {  
    batch.insert(  
      _categoriesTable,  
      category.toInsertMap(),  
      conflictAlgorithm: ConflictAlgorithm.ignore,  
    );  
  }  
  await batch.commit(noResult: true);  
}
```

Using `batch.commit(noResult: true)` avoids allocating result lists for inserts where the returned row IDs are not needed, giving a small performance benefit.

5. CRUD Operations

5.1 Create — insertNote

New notes are inserted with `db.insert()`, which maps directly to SQL INSERT. The method returns the auto-assigned integer id:

```
Future<int> insertNote(Note note) async {
    final db = await database;
    return db.insert(_notesTable, note.toInsertMap());
}

// toInsertMap() returns:
// { title, content, category_id, created_at, updated_at }
// id is intentionally excluded - SQLite assigns it via AUTOINCREMENT
```

5.2 Read — getNotes (with JOIN, WHERE, ORDER BY, LIMIT, OFFSET)

`getNotes()` is the most sophisticated method, dynamically building a WHERE clause and executing a rawQuery JOIN to resolve the category name in a single round-trip:

```
SELECT
    n.id, n.title, n.content, n.category_id,
    n.created_at, n.updated_at,
    c.name AS category_name
FROM notes n
LEFT JOIN categories c ON c.id = n.category_id
[WHERE (n.title LIKE ? OR n.content LIKE ?) [AND n.category_id = ?]]
ORDER BY n.updated_at DESC, n.id DESC
LIMIT ? OFFSET ?
```

The WHERE clause is **dynamically constructed** by accumulating clause fragments and arguments into lists, then joining with AND. This avoids string interpolation vulnerabilities and keeps the code readable. Parameters are passed as positional arguments to prevent SQL injection.

5.3 Read — getNoteById

Fetches a single note by primary key, also performing the same LEFT JOIN to return the category name alongside the note data:

```
Future<Note?> getNoteById(int id) async {
```



```
final rows = await db.rawQuery("""
    SELECT n.*, c.name AS category_name
    FROM notes n
    LEFT JOIN categories c ON c.id = n.category_id
    WHERE n.id = ? LIMIT 1
    """, [id]);
return rows.isEmpty ? null : Note.fromMap(rows.first);
}
```

5.4 Update — updateNote

Uses `db.update()` with a `WHERE` clause scoped to the note's id. Only the mutable fields (title, content, category_id, updated_at) are written — created_at is preserved:

```
Future<int> updateNote(Note note) async {
    final db = await database;
    return db.update(
        _notesTable,
        note.toUpdateMap(), // excludes id and created_at
        where: 'id = ?',
        whereArgs: [note.id],
    );
}
```

5.5 Delete — deleteNote

Deletes by primary key and returns the number of rows affected (0 if not found, 1 on success):

```
Future<int> deleteNote(int id) async {
    final db = await database;
    return db.delete(_notesTable, where: 'id = ?', whereArgs: [id]);
}
```

The `ON DELETE RESTRICT` foreign key constraint means attempting to delete a category that has notes will fail at the database level, protecting referential integrity.

6. Data Models

6.1 Note Model

The Note class is immutable (all fields are final), uses const constructors for compile-time efficiency, and follows a clean serialization contract:

Field	Description
id	int? — nullable; null before insertion, assigned by SQLite AUTOINCREMENT
title	String — required; validated non-empty in UI form
content	String — required; multi-line text body
categoryId	int — foreign key referencing categories.id
categoryName	String? — nullable; populated by JOIN, absent in raw rows
createdAt	DateTime — ISO-8601 TEXT in DB; parsed on fromMap
updatedAt	DateTime — updated on every edit; used for default sort order

copyWith() provides an immutable update pattern — the UI creates a modified copy of a note rather than mutating the original object, keeping state predictable.

6.2 NoteCategory Model

A lightweight model for the categories lookup table:

```
class NoteCategory {
  const NoteCategory({this.id, required this.name});
  final int? id;
  final String name;
  Map<String, dynamic> toInsertMap() => {'name': name};
  factory NoteCategory.fromMap(Map<String, dynamic> map) =>
    NoteCategory(id: map['id'] as int?, name: map['name'] as String);
}
```

6.3 Type Safety

All fromMap() factories use explicit casts (as int?, as String, etc.) rather than dynamic access. This surfaces type mismatches as Dart errors at runtime rather than silent null dereferences. DateTime fields are round-tripped through ISO-8601 strings using DateTime.parse() and toIso8601String(), preserving microsecond precision.

7.1 Screen Layout

The entire app lives on a single NotesPage screen, keeping navigation minimal and focus on the database concepts:

Component	Purpose
Header card	Shows live counts of notes and categories loaded from the DB
Search field	Full-text search across title and content; clears with X button
Category chips	Horizontal scroll row; All chip plus one chip per DB category
Notes list	SliverList with Card per note showing title, category, date, and a 4-line content preview
Note card actions	PopupMenuButton (top-right) and inline TextButton row for Edit / Delete
FAB	FloatingActionButton.extended opens the NoteEditorSheet
Bottom sheet editor	Modal sheet with Form, title field, category dropdown, content field, and Save button

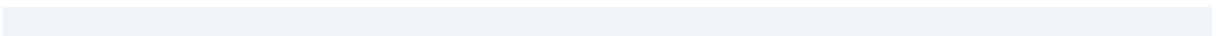
7.2 State Management

The app uses straightforward setState() with async/await — no external state management library is needed for a single-screen app of this scope. Key patterns used:

- `_isLoading` guard: Shows CircularProgressIndicator during initial DB read, avoiding flash of empty state.
- `_reloadNotes()` called after every mutation (insert, update, delete) to keep the list in sync with the database.
- 250 ms debounce on search input prevents a DB query on every keystroke, reducing SQLite I/O.
- `mounted` checks before all setState() calls prevent exceptions if the widget is disposed during an async gap.
- RefreshIndicator on CustomScrollView allows pull-to-refresh as a manual reload trigger.

7.3 NoteEditorSheet

The bottom sheet handles both create and edit modes via the optional note parameter. When note is null the sheet creates; when set it pre-populates fields and calls updateNote on save:



```
if (widget.note == null) {  
  await widget.databaseHelper.insertNote(payload);  
} else {  
  await widget.databaseHelper.updateNote(payload);  
}
```

The form uses `GlobalKey<FormState>` with `TextFormField` validators requiring non-empty title and content. The `DropDownButtonFormField` is pre-seeded from the categories list loaded on app startup.

8. Cross-Platform Database Initialization

8.1 The Problem

sqlite works out of the box on Android and iOS because those platforms provide a native SQLite shared library. On desktop (Windows, macOS, Linux), sqlite requires the `sqlite_common_ffi` package to load a bundled SQLite shared object via Dart FFI.

8.2 Conditional Import Pattern

The app uses Dart's conditional import syntax to select the correct initializer at compile time, with zero runtime overhead:

```
// database_initializer.dart
import 'database_initializer_stub.dart'
    if (dart.library.io) 'database_initializer_io.dart';

void initializeDatabase() => initializeDatabaseImpl();
```

```
// database_initializer_io.dart - desktop
void initializeDatabaseImpl() {
  if (Platform.isWindows || Platform.isLinux || Platform.isMacOS) {
    sqliteFfiInit();
    databaseFactory = databaseFactoryFfi;
  }
}
```

```
// database_initializer_stub.dart - mobile/web (no-op)
void initializeDatabaseImpl() {}
```

This pattern is idiomatic Dart: the stub file satisfies the import on platforms where `dart.library.io` is unavailable (e.g., web), and the io implementation handles all three desktop operating systems.

Why call this in `main()` not in `DatabaseHelper`?

The `databaseFactory` global must be set before the first `openDatabase()` call. Placing initialization in `main()` before `runApp()` guarantees this regardless of when `DatabaseHelper` is first accessed.

9. SQL Topics Covered

9.1 Complete SQL Reference

The table below catalogs every SQL feature used in the app, the method where it appears, and a representative snippet:

SQL Feature	Method	Example
CREATE TABLE	<code>_createDatabase()</code>	CREATE TABLE notes (id INTEGER PRIMARY KEY AUTOINCREMENT, ...)
FOREIGN KEY	<code>_createDatabase()</code>	FOREIGN KEY (category_id) REFERENCES categories (id) ON DELETE RESTRICT
PRAGMA	<code>_configureDatabase()</code>	PRAGMA foreign_keys = ON
INSERT	<code>insertNote(),</code> <code>insertCategory()</code>	<code>db.insert(_notesTable, note.toInsertMap())</code>
Batch INSERT	<code>_seedCategories()</code>	<code>db.batch()</code> with multiple <code>batch.insert()</code> calls
SELECT *	<code>getCategories()</code>	<code>db.query('categories', orderBy: 'name ASC')</code>
SELECT with JOIN	<code>getNotes(),</code> <code>getNoteById()</code>	SELECT n.*, c.name AS category_name FROM notes n LEFT JOIN categories c ON c.id = n.category_id
WHERE (LIKE)	<code>getNotes()</code>	WHERE (n.title LIKE ? OR n.content LIKE ?)
WHERE (=)	<code>getNotes(),</code> <code>getNoteById()</code>	WHERE n.category_id = ? / WHERE n.id = ?
ORDER BY	<code>getNotes()</code>	ORDER BY n.updated_at DESC, n.id DESC
LIMIT / OFFSET	<code>getNotes()</code>	LIMIT ? OFFSET ? (default 100 / 0)
UPDATE	<code>updateNote()</code>	<code>db.update(_notesTable, note.toUpdateMap(), where: 'id = ?')</code>
DELETE	<code>deleteNote()</code>	<code>db.delete(_notesTable, where: 'id = ?', whereArgs: [id])</code>

9.2 Dynamic WHERE Clause Construction

`getNotes()` builds the WHERE clause dynamically — a technique directly applicable to real-world search and filter screens:

```
final whereClauses = <String>[];
final whereArgs = <Object?>[];

if (searchQuery != null && searchQuery.trim().isNotEmpty) {
```

```
        whereClauses.add('n.title LIKE ? OR n.content LIKE ?');
        whereArgs.addAll(['%$searchQuery%', '%$searchQuery%']);
    }
    if (categoryId != null) {
        whereClauses.add('n.category_id = ?');
        whereArgs.add(categoryId);
    }

    final whereSql = whereClauses.isEmpty ? '' : 'WHERE ${whereClauses.join(' AND ')}';
```

10. Package Dependencies

10.1 Production Dependencies

Package	Purpose
sqlite ^2.4.2+1	Primary SQLite plugin for Flutter. Provides Database, openDatabase(), ConflictAlgorithm, and all query helpers. Targets Android and iOS natively.
path ^1.9.1	Pure-Dart path manipulation. Used to call path.join(databasePath, dbName) for a platform-safe file path.
sqlite_common_ffi ^2.4.0+3	Extends sqlite to Windows, macOS, and Linux by loading SQLite via Dart FFI. Provides sqliteFfilnit() and databaseFactoryFfi.
cupertino_icons ^1.0.8	Flutter standard icon pack (included by default, not actively used in this app).

10.2 Dev Dependencies

Package	Purpose
flutter_test	Flutter testing framework. widget_test.dart contains a smoke test verifying the app builds.
flutter_lints ^6.0.0	Recommended Dart lint rules enforcing consistent code style.

11. Learning Outcomes

11.1 Concepts Demonstrated

Working through this app provides hands-on experience with the following concepts from the learning module:

1. DatabaseHelper singleton with lazy database initialization
2. Platform-conditional imports for cross-platform SQLite setup
3. SQL DDL: CREATE TABLE with PRIMARY KEY, AUTOINCREMENT, NOT NULL, UNIQUE, and FOREIGN KEY
4. SQL DML: INSERT, SELECT, UPDATE, DELETE with parameterized queries
5. LEFT JOIN to hydrate model objects with related data in a single query
6. Dynamic WHERE clause construction for search and filter
7. ORDER BY with multiple sort keys; LIMIT and OFFSET for pagination-ready queries
8. Batch operations for efficient bulk inserts
9. Immutable Dart models with toMap/fromMap serialization
10. Flutter UI patterns: StatefulWidget, setState, async/await, debouncing, mounted guards

11.2 Alignment with Learning Module Tasks

Task	Coverage
Watch: SQLite tutorials (3 hrs)	App serves as a working reference implementation to follow along with tutorials
Install sqflite and path packages	pubspec.yaml includes both with correct version constraints
Read sqflite docs	All sqflite APIs used match current pub.dev documentation patterns
Setup database: DatabaseHelper class, initialize DB, create tables, path handling	database_helper.dart implements all four sub-tasks
Learn SQL operations: CREATE, INSERT, SELECT, UPDATE, DELETE, WHERE	All six SQL operations present; see Section 9
Implement CRUD: create, read all, read single, update, delete records	Five DatabaseHelper methods cover each operation
Handle data models: model classes, toMap, fromMap, type safety	note.dart and note_category.dart implement all requirements

Task	Coverage
Query data: filter with WHERE, ORDER BY, LIMIT/OFFSET, JOIN queries	getNotes() demonstrates all four query features
Build database app: Notes app with CRUD, search, persistence	sqlite_app is exactly this deliverable

12. Potential Improvements

12.1 Functional Enhancements

- Database migrations: add a version-aware onUpgrade callback to DatabaseHelper to handle schema evolution without losing data.
- Pagination: getNotes() already accepts limit and offset parameters; wire up infinite scrolling in the UI to load notes in batches.
- Category management: add UI to create and rename categories (the insertCategory method is already present in DatabaseHelper).
- Note archiving: add an archived INTEGER column and toggle notes between active and archived states.
- Export: serialize notes to JSON or CSV using the existing toInsertMap() methods and share via the platform share sheet.

12.2 Technical Enhancements

- Repository pattern: introduce a NoteRepository interface between the UI and DatabaseHelper to facilitate unit testing with a mock repository.
- Riverpod or Bloc: replace setState() with a proper state management solution for scalability beyond a single screen.
- Full-text search (FTS5): use CREATE VIRTUAL TABLE with FTS5 for more performant and feature-rich text search on large note sets.
- Index on category_id: add CREATE INDEX idx_notes_category ON notes(category_id) to speed up the category filter query.
- Widget tests: expand test coverage with a DatabaseHelper mock and pump the NotesPage widget through create/edit/delete flows.

13. Setup & Run Guide

13.1 Prerequisites

- Flutter SDK installed and on PATH (flutter doctor should pass)
- Dart SDK ^3.11.5 (bundled with Flutter)
- A connected device or emulator, or a desktop target (Windows / macOS / Linux)

13.2 Steps

11. Clone or extract the `sqlite_app` project folder.
12. Open a terminal in the project root.
13. Fetch dependencies: `flutter pub get`
14. Run the app: `flutter run`
15. On first launch, four default categories are automatically seeded into the database.
16. Tap Add Note to create a note; use the search field and category chips to filter.

13.3 Desktop Run

For Windows, macOS, or Linux the `sqlite_common_ffi` initialization is handled automatically by `database_initializer_io.dart`. No additional configuration is required:

```
flutter run -d windows    # Windows
flutter run -d macos      # macOS
flutter run -d linux       # Linux
```

14. Conclusion

The SQLite Notes App is a complete, well-structured implementation of all Flutter SQLite learning objectives. It demonstrates practical database skills — from schema design and CRUD operations to JOIN queries and dynamic search — within a production-quality Flutter codebase.

The codebase is notable for several **best practices**: the singleton DatabaseHelper centralizes all SQL in one place; immutable model classes with explicit casts guard against type errors; the conditional import pattern cleanly handles platform differences; and the UI uses debouncing, mounted checks, and loading guards that reflect real-world Flutter development habits.

As a learning deliverable the app goes beyond the minimum requirements — it not only exercises all nine required SQL features but also demonstrates advanced patterns like batch inserts, dynamic WHERE construction, and LEFT JOIN in raw queries. It forms a solid foundation for the enhancements listed in Section 12 and serves as a practical reference for any Flutter developer working with local SQLite storage.

Repository Summary

App: `sqlite_app` | Language: Dart 3 / Flutter | DB: SQLite (sqflite) | Tables: 2 | CRUD methods: 5
| SQL features: 13 | Platforms: Android, iOS, Windows, macOS, Linux